

Stage 27j frozen interface

Rust + CUDA C++

Distributed FX correlator

多 F-engine 板分布式 X-engine 参考设计

按频率块分布、按 sample0 聚合、流水线相关与可见度持久化

本版设计以“**每个 X-engine 实例拥有若干固定 256-channel block，并从全部 F-engine 板接收这些频率块**”为核心。

对于 B 块 F-engine，总输入数为 $N=8B$ ；同一频率块只有在对应 sample0 的 B 个板包完成聚合或到达超时水位线后，才进入 GPU XMAC 流水线。

版本 v0.2

设计输入：Stage 27j F-engine UDP 接口冻结说明

目标实现：Rust 数据面与控制面 + CUDA C++ 相关计算核心

目录

1. 设计结论与边界	10. Rust + CUDA C++ 软件划分
2. Stage 27j 接口摘要	11. 可见度输出队列与 Kafka 判断
3. 分布式总体架构	12. 控制面与运行参数
4. 参数化容量模型	13. 监控、故障与恢复
5. X-engine 实例内部流水线	14. B=4、两节点参考配置
6. UDP 接收、重排与多板聚合	15. 实施与验收路线
7. 早到、晚到、乱序、重复与丢包	附录 A. 关键公式
8. GPU 数据布局与 XMAC	附录 B. 数据结构草案
9. 积分、频率平均与科学标志	

1. 设计结论与边界

核心拓扑：每块 F-engine 将 block 0..15 按控制面配置发往对应 X-engine 节点；每个 X-engine 实例只拥有部分 block，但必须从全部 B 块 F-engine 收到这些 block。实例在本地形成 `[time][channel][global_input]`，然后完成该频率分片的全 N 输入相关。

以 B=4、两台 X-engine 节点为例：

- 节点 A 拥有 block 0..7，即 channel 0..2047，监听 UDP port 4308..4315。
- 节点 B 拥有 block 8..15，即 channel 2048..4095，监听 UDP port 4316..4323。
- 四块 F-engine 都向节点 A 发送低半频带，向节点 B 发送高半频带。
- 节点 A/B 不需要交换原始 F-engine 数据；每个节点独立完成自己的频率范围。

同步边界：当前多板同步和跨板固定延迟尚未验证。因此软件必须同时支持 `engineering_mode` 和 `science_mode`。进入 `science_mode` 前，控制面必须确认 `board_id` 集合、`sample0` 对齐规则、固定整数延迟和分数延迟校准均有效。

1.1 不在本设计范围内

本文不重新定义 Stage 27j F-engine 包格式，也不设计 F-engine 端 PFB。本设计从交换机后的 UDP 接收开始，到可见度 chunk 持久化结束。最终成像、全阵列自校准和 Measurement Set 完整生成可作为后续层。

2. Stage 27j 接口摘要

项目	冻结值	X-engine 含义
每板输入	8 路复数 IQ16	控制面将 board_id 映射到 8 个 global input。
有效复采样率	122.88 MS/s	4096 点 PFB 后每 channel 30 k spectrum/s。
频率划分	16 flows × 256 channels	天然作为分布式 X-engine 的最小所有权单位。
UDP port	4308..4323	port 4308+b 对应 block b，chan0=256b。
每包	128B header + 8192B payload	每包代表一个 board、一个 sample0、一个 256-channel block。
payload	[channel=256][input=8][l16,Q16]	little-endian；同一 channel 内 8 路本板输入连续。
速率	480 kpps、31.9488 Gbps/board	每 block 每板 30 kpps、约 1.9968 Gbps。
时间键	sample0，原始 245.76 MHz sample index	相邻 spectrum 的预期 sample0 增量为 8192。
seq/frame	每 packet 加一；同一 port delta=16	用于诊断丢包；多板 join 的主键使用 sample0 + block。
可靠性	UDP 无重传，checksum/header_crc 为 0	必须由 X-engine 进行长度、字段、一致性和水位线判断。

3. 分布式总体架构

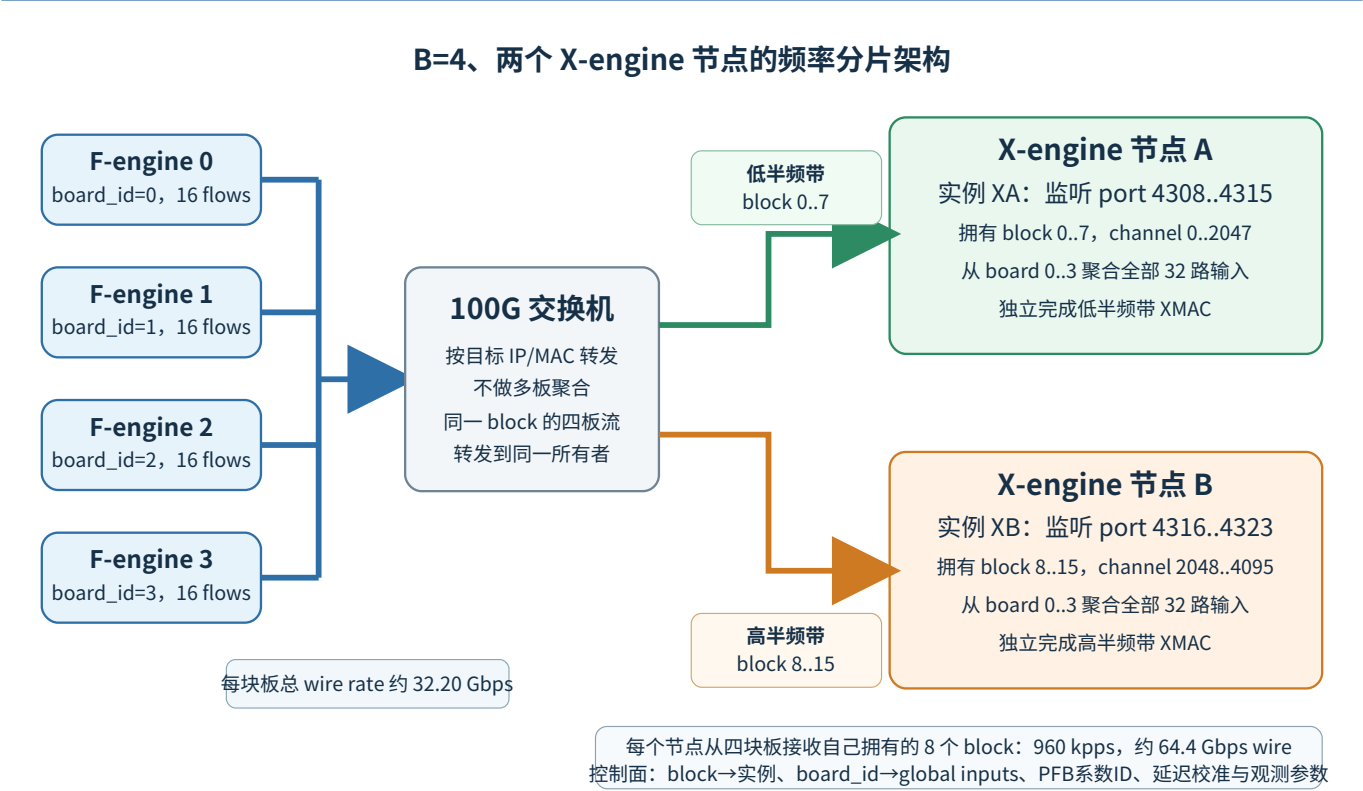


图 3-1。交换机只负责转发；频率块的所有权由控制面配置。一个 X-engine 实例的输入必须覆盖全部 F-engine 板，但只覆盖自身拥有的频率块。

3.1 一般化定义

设 F-engine 板数为 B ，总复输入数 $N=8B$ 。系统有 16 个固定频率 block。每个 X-engine 实例 x 拥有集合 $S_x \subseteq \{0..15\}$ ，集合彼此不重叠且并集覆盖所有需要计算的 block。

```
owner(block_index) -> xengine_instance
expected_boards      = {board_id_0, ..., board_id_(B-1)}
global_input_map     = board_id, local_input -> global_input 0..N-1
```

同一 port 号可以出现在不同主机上，因为目标 IP 不同。如果同一主机运行多个实例，建议给每个实例分配不重叠的 port 集合或独立本地 IP，避免热路径上的 socket 共享和二次分发。

4. 参数化容量模型

一个 block、一个 board 的固定流量为：

```
每 block 每 board: 30,000 packet/s × 8,320 byte × 8 = 1.9968 Gbps T510 UDP payload。
按冻结接口给出的 wire rate，约为 2.0125 Gbps Ethernet wire。
```

若一个实例拥有 K 个 block，则：

packet_rate_instance	= 30,000 × B × K	packet/s
app_rate_instance	= 1.9968 × B × K	Gbps
wire_rate_instance	≈ 2.0125 × B × K	Gbps
channels_instance	= 256 × K	
inputs_instance	= N = 8B	
baselines_unique	= N(N+1)/2	

示例	B	K/实例	pps/实例	app rate	wire rate
4板、2节点均分	4	8	960 kpps	63.8976 Gbps	约 64.4 Gbps
4板、4节点均分	4	4	480 kpps	31.9488 Gbps	约 32.2 Gbps
8板、4节点均分	8	4	960 kpps	63.8976 Gbps	约 64.4 Gbps

建议按 100G NIC 的持续工程负载保留余量。以约 80 Gbps 作为首版规划水位时，最大 block 数可由 floor(80 / (2.0125B)) 粗略估计，再结合 pps、CPU 和 NIC 队列实测调整。

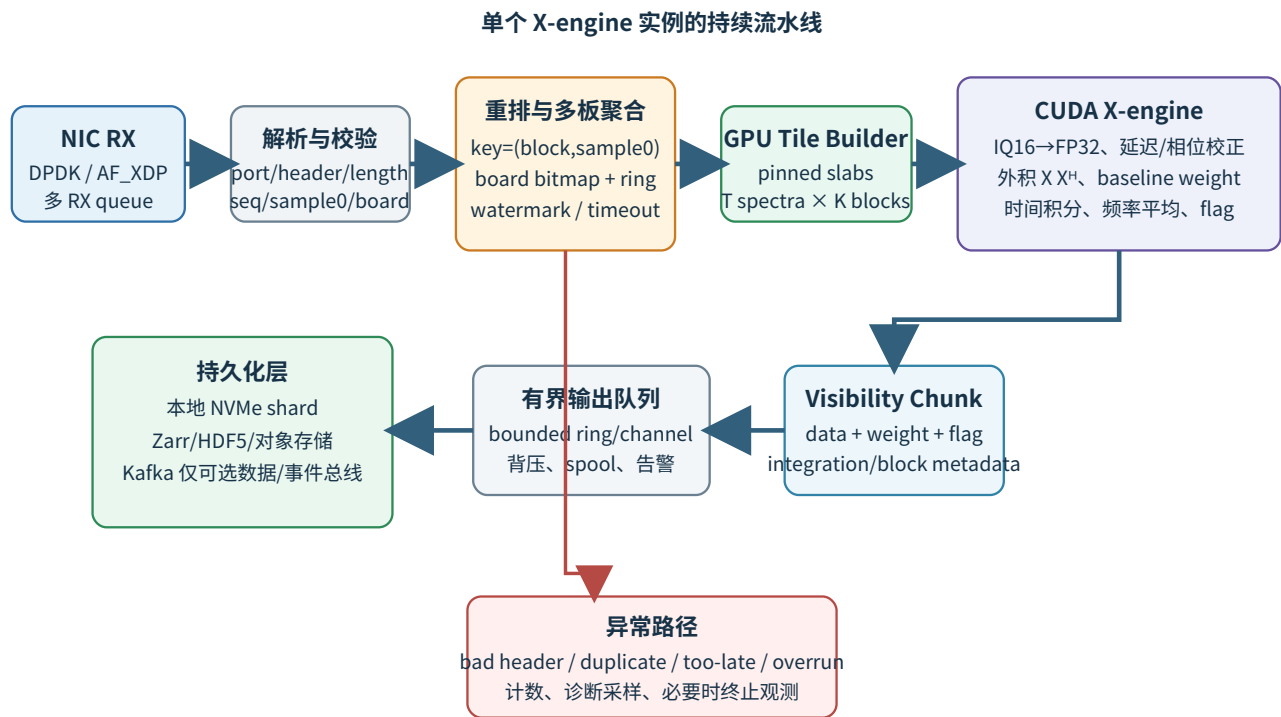
4.1 B=4、K=8 的 GPU 计算量

N=32，unique baseline 数为 528。一个节点负责 2048 channel：

complex CMAC/s	= 2048 × 528 × 30,000 = 32.44032 GCMAC/s
按 8 FLOP/complex CMAC 估算	≈ 259.52 GFLOP/s

该计算量通常不是首要瓶颈；更难的是 960 kpps 接收、多板 join、内存搬运、无丢包流水线和稳定落盘。

5. X-engine 实例内部流水线



所有级之间使用有界队列；不得使用无界缓存掩盖吞吐不足。控制面异步，热路径使用固定线程、固定内存和预分配对象。

图 5-1。接收、聚合、GPU 和写盘并行运行；每一级只通过有界队列交换所有权，避免互相阻塞。

5.1 推荐并行粒度

阶段	推荐粒度	说明
NIC RX	每 RX queue 一个固定线程	按 UDP port/RSS 把 block 分散到队列；线程只做最小解析和投递。
Assembler	每 block 一个 shard，或每 2-4 blocks 一个 shard	同一 block 的 sample0 ring 只由一个线程写，减少锁。
GPU batch	$T=32/64/128 \text{ spectra} \times K \text{ blocks}$	以 64 spectra 为首版基线，降低 kernel launch 和 H2D 频率。
Writer	每实例一个 writer + 多 sink 后端	先写本机 shard，避免多个节点竞争同一个单文件锁。

6. UDP 接收、重排与多板聚合

6.1 正确的 join key

同一个完整 spectrum 在单板内的 16 个 block 共享相同 `sample0`，但 `frame_id` 和 `seq_no` 是逐 packet 增长的。因此跨板聚合的主键应为：

```
JoinKey = (observation_id, block_index, corrected_sample0)
```

其中：

```
block_index = dst_port - 4308
expected sample0 step = 245.76e6 / 30,000 = 8192 raw-sample ticks
corrected_sample0 = packet.sample0 + board_sample0_offset_raw[board_id]
```

`frame_id`、`seq_no` 用于一致性诊断；不能把它们作为跨板的唯一时间键，因为不同板启动计数的绝对值未必一致。

6.2 聚合槽结构

```
AssemblySlot {
    key: JoinKey,
    board_bitmap: B bits,
    packet_ptr[B],           // 每板一个 8192B payload 引用
    arrival_time[B],
    valid_input_mask[N],    // 每板到达即置对应 8 bits
    state: EMPTY | OPEN | COMPLETE | EXPIRED | CONSUMED
}
```

当 `popcount(board_bitmap)==B` 时立即完成。完成槽不需要等待同一板的其他 block，因为实例只负责自己拥有的 block，每个 block 独立送入 GPU tile。

6.3 环形重排窗口

每个 block 使用固定长度 R 的 `sample0` ring。槽号可以由 spectrum index 计算：

```
spectrum_index = corrected_sample0 / 8192
slot_index      = spectrum_index mod R
```

槽内仍保存完整 `sample0`，以检测 ring wrap 和陈旧包。建议首版：

参数	建议起点	B=4、K=8 的内存量级
R=128 spectra	约 4.27 ms 网络重排窗口	128×8×4×8192 ≈ 32 MiB payload
R=256 spectra	约 8.53 ms 网络重排窗口	约 64 MiB payload

参数	建议起点	B=4、K=8 的内存量级
R=512 spectra	约 17.07 ms	约 128 MiB payload

这类缓存应使用预分配 hugepage/pinned memory，不要逐包分配。

7. 早到、晚到、乱序、重复与丢包

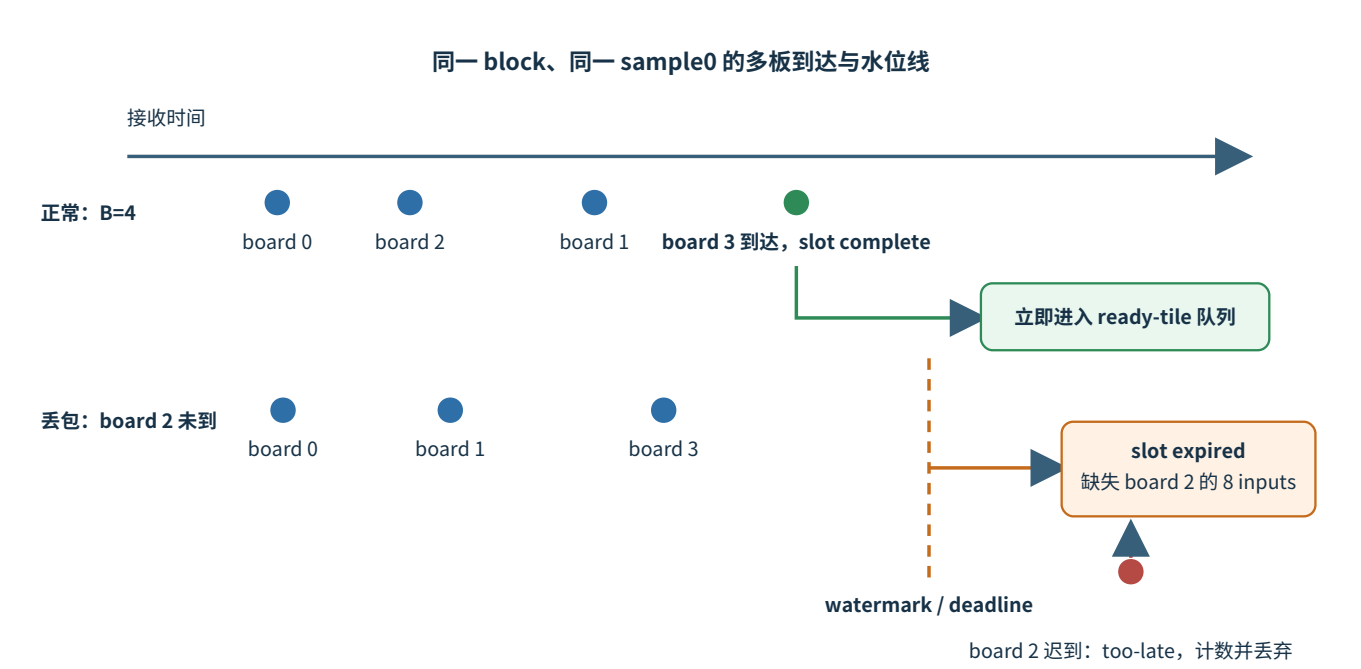


图 7-1。早到包只占据 ring slot；最后一块板到达即完成。水位线后仍缺失则按策略结算，之后到达的包只能计为 too-late。

7.1 水位线算法

```
max_seen_sample0[block] = max(max_seen_sample0[block], corrected_sample0)
watermark = max_seen_sample0[block] - reorder_window_spectra * 8192

for any OPEN slot with slot.sample0 <= watermark:
    finalize_as_complete_or_missing(slot)
```

水位线基于数据进度，而不是主机 wall-clock。可另外设置 wall-clock watchdog，防止整个流停止时 OPEN slot 永不结算。

7.2 UDP 异常处理表

情况	检测	处理
乱序/早到	sample0 在窗口内但不是当前最小值	写入对应 ring slot，正常等待。
同一板重复包	slot 的 board bit 已置位	比较 header；相同则 drop duplicate，不同则报 protocol conflict。

情况	检测	处理
包丢失	slot 到 watermark 仍缺 board bit	按 baseline-aware 或 strict-block 策略结算，并累加 loss。
迟到包	对应 slot 已 EXPIRED/CONSUMED	丢弃；不得修改已经输出的可见度。
seq gap	同 board、同 port 的 seq delta 不是 16	记录诊断；最终以 sample0 join 结果为准。
sample0 步长异常	同流不满足 +8192	标记 discontinuity，结束当前 integration 或进入新 epoch。
字段不一致	chan0、block、ninput、payload size 错误	立即 drop 并计 protocol_error。

7.3 推荐的科学丢包策略

推荐 baseline-aware：缺失一个板包时，只把该板对应 8 路输入标记为无效。GPU 对无效输入置零，同时对每条 baseline 累加独立的有效样本计数 weight。这样仍能保留已到达板之间的相关数据。

```
valid_input_mask: N bits
for baseline (i,j):
    if valid[i] && valid[j]:
        V[i,j] += Xi * conj(Xj)
        W[i,j] += 1

integration output:
    Vnorm[i,j] = V[i,j] / W[i,j]  if W>0
    FLAG[i,j] = 1                  if W below configured threshold
```

可配置一个更严格的 `strict_block` 模式：任意板缺包则整个 block/time 丢弃。该模式更简单，适合初期验证；正式观测更建议 baseline-aware。

8. GPU 数据布局与 XMAC

8.1 聚合后的逻辑数据

单板 packet 的 payload 是 `[channel=256][local_input=8][IQ16]`。同一 slot 收齐 B 个包后，逻辑上形成：

```
X[time][channel=256][global_input=N]
```

首版不必在 CPU 上把 B 个 payload 完全拷贝成连续 N 输入。更高效的方式是：

1. Rust 将每个板的 8192B payload 放入固定 board slot。
2. tile 完成后把 `[time][block][board][channel][local_input]` 批量 H2D。
3. CUDA gather/convert kernel 使用控制面映射表将其转成 `[block][channel][time][global_input]` FP32。

这样可以减少每包 CPU scatter copy，并把 endian、IQ16→FP32、board 映射和 validity mask 合并为一个 GPU kernel。

8.2 推荐 tile

设一个 GPU tile 含 T 个 spectrum-time。B=4、K=8、T=64 时：

```
packets/tile = B × K × T = 4 × 8 × 64 = 2048 packets
payload/tile = 2048 × 8192B = 16 MiB
tile duration = 64 / 30,000 ≈ 2.133 ms
```

首版建议 T 可配置为 32、64、128。T 太小会增加 launch 开销，太大则增加丢包等待延迟和缓冲。

8.3 相关计算

对每个频率 channel c 和时间 tile：

```
V_ij[c] += sum_t X[t,c,i] * conj(X[t,c,j])
0 <= i <= j < N
```

unique baseline 数：

```
M = N(N+1)/2 = 8B(8B+1)/2
```

CUDA 输出可使用 packed upper-triangle baseline order。建议固定索引：

```
baseline_index(i,j) = i*N - i*(i-1)/2 + (j-i), 0 <= i <= j < N
```

8.4 延迟和相位校正

多板时钟“锁定”不代表链路固定延迟为零。正式 XMAC 前应支持每 input 的校正：

$$Xcorr_i(c,t) = X_i(c,t) * G_i(c,t) * \exp(-j \ 2\pi \ f_c \ \tau_i(t))$$

其中控制面提供：

- `board_sample0_offset_raw`：整数 raw-sample 对齐，仅用于 join key 校正。
- `fixed_delay_seconds[input]`：电缆/板卡固定延迟。
- `fractional_delay_seconds[input]`：通过 channel phase slope 校正。
- `complex_gain[input][channel]`：bandpass/静态相位，可按版本更新。

若跨板固定延迟尚未验证，输出必须携带 `sync_valid=false` 或 calibration state，不应被下游误认为可直接成像的科学可见度。

8.5 数值精度

阶段	建议格式	原因
输入	signed IQ16	冻结接口。
校正/XMAC	FP32 complex	与 GB10/CUDA 目标一致，吞吐充分。
短 tile 累加	FP32 pairwise/hierarchical	避免每样本全局 atomic。
长 integration	可选 FP32 hierarchical 或 FP64 final accumulator	按动态范围和性能实测选择。
输出	complex64 + u32/u64 weight	适合科学存储和后续校准。

9. 积分、频率平均与科学标志

9.1 时间积分

每 channel 的 spectrum-time rate 固定为 30 kHz。积分时间应转换成整数 spectrum 数：

$$\begin{aligned} S_int &= \text{integration_spectra} \\ T_int &= S_int / 30,000 \text{ seconds} \end{aligned}$$

配置接口应以 `integration_spectra` 为最终权威，用户可以输入秒数，由控制面校验并转换。典型值：

integration_spectra	实际积分时间	用途
300	10 ms	快速瞬变/调试，输出率极高。
3000	100 ms	快速监控。
30000	1 s	常规首版科学输出。
300000	10 s	低输出率，但要检查时变相位和 time smearing。

integration 边界建议按 corrected sample0 对齐：

```
integration_id = floor((sample0 - observation_sample0) / (8192 * S_int))
```

当 sample0 discontinuity 或配置版本变化时，必须强制 flush 当前 integration，并以新 epoch 重启。

9.2 频率平均

freq_avg_factor=F 表示在相关和校正之后，将连续 F 个 PFB channel 的可见度按 weight 加权平均。首版建议只允许 F 为 256 的因数：

```
F ∈ {1,2,4,8,16,32,64,128,256}
output_channels_per_block = 256 / F
```

应在完成 per-channel delay/phase correction 后再平均，避免不同 channel 的相位斜率相互抵消。平均时同步合并：

```
Vavg = sum(V[c] * W[c]) / sum(W[c])
Wavg = sum(W[c])
FLAG = threshold(sum(W[c]), expected_weight)
```

9.3 运行时可调参数

参数	范围/含义	生效规则
owned_blocks	实例拥有的 block 集合	观测开始前固定；改变需重建 RX 和 buffer。
expected_boards	参与板集合 B	观测开始前固定。
integration_spectra	时间积分长度	仅在 integration 边界切换。
freq_avg_factor	频率平均因子	仅在 output chunk 边界切换。
reorder_window_spectra	乱序等待窗口	可热更新，但要保证 ring 容量。
loss_policy	baseline-aware / strict-block	integration 边界生效。
min_valid_fraction	baseline 最低有效样本比例	输出 FLAG 判定。
accumulation_mode	fp32_pairwise / fp64_final	观测开始前固定。
calibration_version	delay/gain 模型版本	切换时 flush integration。

10. Rust + CUDA C++ 软件划分

10.1 Rust 负责

- NIC 初始化、DPDK/AF_XDP RX、RX queue 与 CPU affinity。
- T510 header 解析和 Stage 27j 协议校验。
- 按 block 分片、sample0 ring、board bitmap 和 watermark。
- 预分配 hugepage/pinned buffer 的生命周期和所有权。
- CUDA C ABI 调用、完成事件轮询、output chunk 排队。
- 控制面 gRPC/HTTP、配置版本、指标和日志。
- 文件/对象存储/Kafka sink 的调度。

热路径建议固定线程和有界 ring，不依赖异步 runtime 调度；控制面和低速 writer 管理可以使用 Tokio。

10.2 CUDA C++ 负责

- IQ16 little-endian 解包、FP32 转换和 board→global input gather。
- validity mask 展开。
- per-input/channel 复增益、固定/分数延迟和 fringe phase。
- 按 channel 批处理的 Hermitian outer product。
- 时间积分、baseline weight 和频率平均。
- VisibilityChunk device buffer 生成和异步 D2H。

10.3 建议的工程目录

```
xengine/  
Cargo.toml  
crates/  
  protocol/      # T510 header、little-endian parser、schema tests  
  rx/            # DPDK/AF_XDP、RSS、RX queues  
  assembler/     # sample0 ring、watermark、loss policy  
  pipeline/      # bounded queues、tile lifecycle  
  control/       # observation config、board registry、block assignment  
  writer/        # local shard、Zarr/HDF5、Kafka event sink  
  metrics/       # Prometheus counters  
cuda/  
  include/xengine_cuda.h  
  src/gather_convert.cu  
  src/delay_phase.cu  
  src/xmac.cu  
  src/integrate_average.cu  
  src/c_api.cpp  
configs/  
tests/  
  packet_replay/  
  loss_reorder/  
  cuda_reference/
```

CUDA 库导出稳定 C ABI，Rust 侧通过 FFI 调用，避免在热路径引入 C++ 对象生命周期跨语言问题。

11. 可见度输出队列与 Kafka 判断

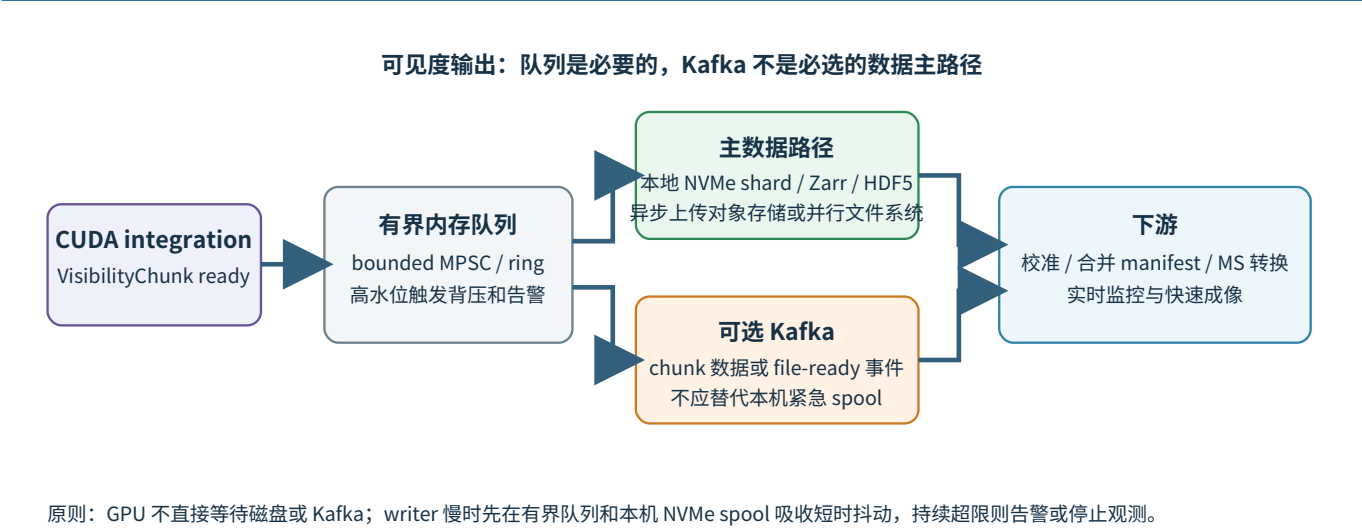


图 11-1。队列是流水线隔离手段；Kafka 是可选的分布式传输/事件组件，不是高吞吐科学数组持久化的唯一答案。

11.1 是否必须使用 Kafka?

必须有队列，但不一定是 Kafka。实例内部至少要有一个有界 VisibilityChunk 队列，把 GPU 与写盘解耦。Kafka 适合：

- 发布 chunk_ready、file_closed、质量统计和观测状态事件。

- 在积分较长、频率平均较大、输出率有限时传输完整 visibility chunk。
- 需要多个实时消费者订阅同一份已积分数据时。

Kafka 不适合作为唯一安全阀：

- 短积分会使可见度输出率急剧上升。
- 消息复制、序列化和 broker retention 增加系统复杂度。
- Kafka 可用性下降时，不能让 GPU 热路径同步等待。

11.2 B=4、N=32 的输出率示例

两节点均分频率，每节点 K=8。无频率平均、complex64、仅计算 528 个 unique baselines：

$$\begin{aligned} \text{visibility bytes/integration/node} &= 2048 \text{ channels} \times 528 \times 8\text{B} \\ &= 8,650,752 \text{ bytes} \approx 8.65 \text{ MB} \end{aligned}$$

积分时间	仅 visibility/node	判断
1 s	约 8.65 MB/s	Kafka 技术上可行，但本地 shard 仍更直接。
100 ms	约 86.5 MB/s	需要谨慎规划 broker、复制和消费者。
10 ms	约 865 MB/s	不建议用 Kafka 作为主要科学数据路径。

加上 weight、flag 和元数据后还会更高。因此推荐默认模式：

$$\begin{aligned} \text{GPU} &\rightarrow \text{bounded queue} \rightarrow \text{local NVMe shard} \rightarrow \text{async uploader} \\ &\quad \quad \quad \hookrightarrow \text{Kafka 发布 metadata / manifest / optional decimated chunks} \end{aligned}$$

12. 控制面与运行参数

12.1 观测配置快照

```
ObservationConfig {
    observation_id,
    expected_board_ids[B],
    board_input_map[board_id][8] -> global_input,
    pfb_coefficient_id[board_id],
    board_sample0_offset_raw[board_id],
    fixed_delay_seconds[N],
    gain_table_version,
    block_owner[16] -> instance_id,
    integration_spectra,
    freq_avg_factor,
    reorder_window_spectra,
    loss_policy,
    min_valid_fraction,
    output_format,
    output_destination
}
```

配置在观测启动时生成不可变 hash，并写入每个 VisibilityChunk。运行中影响物理意义的参数变化必须切 integration 并生成新 config version。

12.2 实例启动准入

- 1. 确认 owned block 与监听 port 一致。
- 2. 确认 expected board_id 无重复，映射覆盖 N=8B。
- 3. 确认所有参与板 PFB coefficient ID 一致，或明确允许并记录差异。
- 4. 在 science_mode 下确认多板 sample0 对齐和延迟校准 valid。
- 5. 预热 NIC RX、hugepage pool、pinned slabs、CUDA buffers 和 writer spool。
- 6. 等待共同 observation epoch/sample0，再打开 integration。

13. 监控、故障与恢复

类别	关键指标	处置
RX	pps/Gbps、NIC drop、RX ring fill、每 port/board 速率	出现持续 drop 时停止观测，不靠扩大无界缓存掩盖。
协议	bad length、bad block、unknown board、seq gap、sample discontinuity	计数并输出诊断样本。
聚合	complete ratio、missing board、late、duplicate、reorder depth	调整 reorder window；检查交换机和板同步。

类别	关键指标	处置
GPU	tile latency、H2D/D2H、kernel time、queue depth	tile 大小和 stream 数可调；防止 ready queue 堆积。
科学质量	baseline valid fraction、auto power、phase continuity、delay residual	超过阈值 flag integration 或退出 science_mode。
Writer	queue depth、NVMe latency、spool usage、upload lag	高水位告警；spool 满则有序停止观测。

13.1 节点失败

频率块只由一个实例拥有时，节点失败会丢失对应频段。首版推荐简单的静态单所有者模型；后续可以增加：

- 热备实例，交换机/F-engine 在故障后切换目标 IP。
- 关键观测时双播某些 block，但会成倍增加 F-engine 和网络负载。
- 由控制面重新分配 block，并以新 observation epoch 启动。

14. B=4、两节点参考配置

项目	节点 A	节点 B
实例 ID	XA	XB
owned blocks	0..7	8..15
channels	0..2047	2048..4095
UDP ports	4308..4315	4316..4323
expected boards	0,1,2,3	0,1,2,3
inputs	32	32
ingress	960 kpps、约 64.4 Gbps wire	相同
GPU tile	64 spectra × 8 blocks × 4 boards ≈ 16 MiB payload	相同
XMAC	约 259.5 GFLOP/s	相同
1s visibility	约 8.65 MB/s，不含 weight/flag	相同

14.1 推荐首版参数

```
reorder_window_spectra = 256      # 8.53 ms
wallclock_timeout_ms   = 20
cuda_tile_spectra      = 64       # 2.133 ms
integration_spectra     = 30000    # 1 s
freq_avg_factor         = 1
loss_policy             = baseline_aware
min_valid_fraction      = 0.99
accumulation_mode        = fp32_pairwise
output                  = local_zarr_shard + kafka_manifest_event
```

这些值是工程起点，不是最终科学参数。应通过交换机抖动测试、丢包注入和真实延迟标定后调整。

15. 实施与验收路线

阶段	目标	通过条件
1. 单板单 block replay	Rust 解析和 sample0 ring	30 kpps、无误解析、seq delta=16 检查正确。
2. 四板单 block	多板 join	乱序、重复、延迟和丢包注入结果符合 bitmap/watermark 策略。
3. 四板八 blocks	节点满 ingress	960 kpps、约 64.4 Gbps wire、NIC/assembler 0 drop。

阶段	目标	通过条件
4. CUDA reference	32 输入 XMAC	与 NumPy/C++ double reference 在容差内一致。
5. 流水线 soak	RX/GPU/writer 并发	数小时运行，所有 bounded queue 水位稳定。
6. 丢包科学验证	baseline-aware weight	缺板时仅相关 baseline 被 flag，其他 baseline 数值正确。
7. 两节点全频带	block 0..15 覆盖	输出 manifest 中频率无空洞、无重叠、integration 边界一致。
8. 多板延迟校准	science_mode 准入	已知 tone/噪声源的跨板相位连续，固定/分数延迟模型可重复。

附录 A. 关键公式

$$N = 8B$$

$$M = N(N+1)/2$$

$$\text{per_block_per_board_pps} = 30,000$$

$$\text{per_block_per_board_app_rate} = 30,000 \times 8,320 \times 8 = 1.9968 \text{ Gbps}$$

$$\text{instance_pps} = 30,000 \times B \times K$$

$$\text{instance_rate} = 1.9968 \times B \times K \text{ Gbps}$$

$$\text{channels_instance} = 256K$$

$$\text{XMAC_FLOP/s} \approx \text{channels_instance} \times M \times 30,000 \times 8$$

$$\text{sample0_step} = 245.76e6 / 30,000 = 8192 \text{ raw ticks}$$

$$\text{output_channels_instance} = 256K / \text{freq_avg_factor}$$

$$\text{visibility_bytes_per_integration} = \text{output_channels_instance} \times M \times 8$$

附录 B. VisibilityChunk 草案

```
VisibilityChunk {
    schema_version,
    observation_id,
    config_hash,
    instance_id,
    block_first,
    block_count,
    chan0,
    nchan_raw,
    freq_avg_factor,
    nchan_output,

    integration_id,
    integration_start_sample0,
    integration_spectra_expected,
    integration_spectra_seen,

    ninput,
    nbaseline,
    baseline_order,
    board_input_map_hash,
    pfb_coefficient_ids,
    calibration_version,
    sync_valid,

    visibility: complex64[nchan_output][nbaseline],
    weight: u32/u64[nchan_output][nbaseline],
    flag: bit/u8[nchan_output][nbaseline],

    packet_loss_by_board,
    late_packet_count,
    duplicate_packet_count,
    sample_discontinuity_count
}
```

参考资料

1. Stage 27j F-engine UDP frozen interface: 用户提供的接口说明, 本设计的协议与速率基线。
2. NVIDIA CUDA C++ Programming Guide: CUDA streams、page-locked memory、异步执行和内存层次的实现依据。
3. Linux Kernel AF_XDP documentation: 高性能用户态 packet RX 的官方接口说明。
4. DPDK Programmer's Guide: poll-mode driver、hugepage、ring 和多队列数据面的实现参考。
5. Apache Kafka documentation: Kafka 作为可选事件/数据总线时的官方语义参考。

本文档取代此前 v0.1 X-engine 草案。关键修正: 以“频率块单所有者 + 全部 F-engine 板聚合”为分布式边界, 并明确 UDP 等待、丢包权重、可调积分/频率平均和可见度持久化策略。